Richard Bird's Algebraic Laws for Program Calculation

A Detailed Survey of the Eight Primary Publications

Compiled from Bird's publications (1984–2020)

March 2026

Contents

Introduction **3**

1. The Promotion and Accumulation Strategies in Transformational Programming (1984) **3**
Context . . . 3
Laws Introduced . . . 3
Significance . . . 4

2. An Introduction to the Theory of Lists (1986) **5**
Context . . . 5
Operations Defined . . . 5
Laws Stated and Proved . . . 5
Significance . . . 6

3. A Calculus of Functions for Program Derivation (1987) **7**
Context . . . 7
Key Contributions . . . 7
Significance . . . 7

4. Lectures on Constructive Functional Programming (1988) **8**
Context . . . 8
Structure . . . 8
Key Applications . . . 9
Significance . . . 9

5. Algebraic Identities for Program Calculation (1989) **10**
Context . . . 10
Laws Used (Numbered Equations 1–13) . . . 10
The MSS Derivation . . . 11
Significance . . . 11

6. The Algebra of Programming (1996) **12**
Context . . . 12
Structure and Scope . . . 12

Significance . . . 13

7. Generic Functional Programming with Types and Relations (1996) 14
Context . . . 14
Key Contributions . . . 14
Significance . . . 14

8. Algorithm Design with Haskell (2020) 15
Context . . . 15
Structure . . . 15
Laws Presented . . . 15
Notation . . . 15
Significance . . . 16

The Evolution of Notation for Fold and Scan 17
Phase 1: APL Origins (1962–1978) . . . 17
Phase 2: Meertens' Algorithmics (1984–1986) . . . 17
Phase 3: The Squiggol Period (1985–1988) . . . 17
Phase 4: The Retreat from Squiggles (1988–1989) . . . 18
Phase 5: Categorical Notation (1993–1997) . . . 18
Phase 6: Plain Haskell (1998–2020) . . . 18
Summary Timeline . . . 19

Summary: Lineage of the Laws 20

Appendix A: Quick Reference of Core Laws (Haskell Notation) 21

Appendix B: Core Laws in BMF Symbolic Notation 22

Introduction

Between 1984 and 2020, Richard Bird developed a body of algebraic laws for transforming functional programs from clear but inefficient specifications into efficient implementations. These laws—concerning map, fold, scan, promotion, accumulation, Horner's Rule, and fusion—form the core of the Bird–Meertens Formalism (BMF), colloquially known as "Squiggol."

This document surveys the **eight primary publications** in which these laws are defined and proved, in chronological order. For each paper we describe the context, the specific laws introduced, their formal statements, and their significance.

---

1. The Promotion and Accumulation Strategies in Transformational Programming (1984)

Citation: R. S. Bird. "The Promotion and Accumulation Strategies in Transformational Programming." *ACM Transactions on Programming Languages and Systems* 6(4):487–504, October 1984.

Addendum: R. S. Bird. "Addendum to 'The Promotion and Accumulation Strategies in Transformational Programming.' " *ACM TOPLAS* 7(3):490–492, July 1985.

Context

This is Bird's first systematic treatment of two named transformation strategies for deriving efficient programs from specifications over list structures. The paper predates the "squiggly" BMF notation; it uses a more conventional mathematical style. It was published in TOPLAS, the premier venue for programming language theory at the time.

Laws Introduced

The Promotion Strategy

The promotion strategy concerns the interaction of a function with the concatenation (or "join") of data structures. In modern terms, this encompasses:

- **Map promotion:** If f is applied element-wise (mapped) over a concatenated structure, it can be "promoted" into the components:

$$\text{map } f \text{ (concat } xss) = \text{concat (map (map } f) \; xss)$$

- **Reduce promotion:** If a reduction (fold) is applied to a concatenated structure, it can be promoted into the components:

$$\text{foldr } (\oplus) \; e \text{ (concat } xss) = \text{foldr } (\oplus) \; e \text{ (map (foldr } (\oplus) \; e) \; xss)$$

These "promotion" laws allow transformations that decompose a computation over a compound structure into computations over its parts.

The Accumulation Strategy

The accumulation strategy introduces the idea of computing all intermediate results of a fold—what we now call a *scan*. The key insight is that independent fold computations over overlapping prefixes (or suffixes) of a list can be replaced by a single left-to-right (or right-to-left) accumulation:

$$\text{map (foldl } (\oplus)\ e) \text{ (inits } xs) = \text{scanl } (\oplus)\ e\ xs$$

This replaces $O(n^2)$ work (one fold per prefix) with $O(n)$ work (one scan).

Significance

This paper established the two core strategies that recur throughout all of Bird's subsequent work. Every derivation in the 1989 paper and beyond uses promotion and accumulation as fundamental moves.

---

2. An Introduction to the Theory of Lists (1986)

Citation: R. S. Bird. "An Introduction to the Theory of Lists." Technical Monograph PRG-56, Programming Research Group, Oxford University, October 1986. Published in M. Broy (ed.), *Logic of Programming and Calculi of Discrete Design*, Springer-Verlag, 1987, pp. 3–42.

Context

This is the **foundational reference** for the entire BMF as applied to lists. Written as lecture notes at Oxford, PRG-56 introduces a complete notation and calculus for list manipulation, defines all the core operations and their algebraic properties, and proves the principal laws. Much of the subsequent work (including the 1989 paper) is a condensation or application of material first presented here.

Operations Defined

The monograph defines, with full types and algebraic properties:

- **Concatenation** ($+\!\!+$): Associative with identity $[\,]$.
- **Map** ($f*$): Applies f to every element. Distributes through concatenation.
- **Reduce** ($\oplus/$): Collapses a list via an associative operator.
- **Filter** ($p\triangleleft$): Selects elements satisfying predicate p.
- **Directed reductions** ($\oplus\leftarrow e$ and $\oplus\rightarrow e$): Left and right folds with a seed value.
- **Accumulations** ($\oplus\leftarrow\!\!\downarrow\ e$ and $\oplus\rightarrow\!\!\downarrow\ e$): Left and right scans, producing all intermediate fold results.
- **Segments:** inits, tails, segs (all initial segments, final segments, and contiguous segments).

Laws Stated and Proved

Map Laws

1. **Map fusion (distributivity over composition):**

$$(f \circ g)* = (f*) \circ (g*)$$

2. **Map distribution over concatenation:**

$$f * (x +\!\!+ y) = (f *\ x) +\!\!+ (f *\ y)$$

3. **Inverse law:** If f is injective with inverse f^{-1}, then $(f*)^{-1} = (f^{-1})*$.

Promotion Laws

4. **Map promotion (join rule for map):**

$$f * \circ \text{concat} = \text{concat} \circ (f*)*$$

5. **Reduce promotion (join rule for reduce):**

$$\oplus/ \circ \text{concat} = \oplus/ \circ (\oplus/)*$$

Duality and Specialisation

6. **Duality lemma:** Left and right folds are related by reversal:

$$\text{foldl } (\oplus)\ e = \text{foldr } (\widetilde{\oplus})\ e \circ \text{reverse}$$

   where $a \widetilde{\oplus} b = b \oplus a$.

7. **Specialisation lemma:** Every list homomorphism can be expressed as either a left or a right reduction.

The Accumulation Lemma (Scan Lemma)

8. **Accumulation lemma:**

$$\text{scanl } (\oplus)\ e = \text{map } (\text{foldl } (\oplus)\ e) \circ \text{inits}$$

   (and a dual for right accumulations and tails).

   This is the defining specification of scan: the k-th element of a left scan is the left fold over the first k elements. Computing map (foldl $(\oplus)$ e) $\circ$ inits takes $O(n^2)$ time; computing scanl $(\oplus)$ e directly takes $O(n)$.

Horner's Rule

9. **Horner's Rule:** If $\otimes$ distributes backwards over $\oplus$ (i.e., $(a \oplus b) \otimes c = (a \otimes c) \oplus (b \otimes c)$), then:

$$\oplus/ \circ \otimes/ * \circ \text{tails} = (@) \rightarrow e$$

   where $e = \text{id}_{\oplus}$ and $a \mathbin{@} b = (a \otimes b) \oplus e$.

   This converts an $O(n^2)$ computation (reduce over each tail, then reduce the results) into an $O(n)$ single right fold.

Homomorphism Lemma

10. **Homomorphism lemma:** Every list homomorphism h (satisfying $h(x +\!\!+ y) = h(x) \oplus h(y)$) can be decomposed as:

$$h = (\oplus/) \circ (f*)$$

   and conversely, every such composition is a homomorphism.

Significance

PRG-56 is the single most important reference for the list-level algebraic laws. All subsequent papers and books by Bird build on definitions and results first stated here. The notation evolved (from squiggles to Miranda to Haskell), but the *laws themselves* are unchanged.

3. A Calculus of Functions for Program Derivation (1987)

Citation: R. S. Bird. "A Calculus of Functions for Program Derivation." Technical Monograph PRG-64, Programming Research Group, Oxford University, December 1987. Published in D. A. Turner (ed.), *Research Topics in Functional Programming*, Addison-Wesley, 1990.

Context

This monograph extends the theory of PRG-56 by developing a more complete calculus: a systematic notation and two general theorems for transforming specifications into efficient implementations. Bird explicitly argues that "an effective calculus of program derivation must be built upon useful general theorems relating certain forms of expression to common patterns of computation."

Key Contributions

Notation

PRG-64 introduces a refined notation for curried functions, functional composition, and operators, aiming for a calculus as usable as integral calculus is for mathematicians. The notation is intermediate between the early squiggles (PRG-56) and the later Miranda/Haskell conventions.

Two General Theorems

The paper states and proves two main theorems:

1. **Theorem 1** concerns the efficient evaluation of expressions of the form $\oplus/ \circ f * \circ g$, where g generates a data structure and $f*$ maps over it. Under suitable conditions (distributivity, associativity), this can be transformed into a directed reduction.
2. **Theorem 2** generalises the first to handle more complex patterns involving nested compositions of reductions and maps.

These theorems subsume the promotion laws, the accumulation lemma, and Horner's Rule as special cases, providing a unified framework for the individual laws of PRG-56.

Application: Run-Length Encoding

The paper demonstrates the calculus by deriving an efficient algorithm for run-length encoding from its specification (the shortest sequence of pairs that decodes to the given input). The derivation proceeds entirely by equational reasoning, applying the general theorems.

Significance

PRG-64 is the most "calculus-like" of Bird's presentations: it argues for and demonstrates a style of derivation modelled on mathematical problem solving (like formal integration), where one applies known theorems rather than ad hoc reasoning. The two general theorems provide a unifying perspective on the individual laws.

---

4. Lectures on Constructive Functional Programming (1988)

Citation: R. S. Bird. "Lectures on Constructive Functional Programming." Technical Monograph PRG-69, Programming Research Group, Oxford University, September 1988. Published in M. Broy (ed.), *Constructive Methods in Computer Science*, Springer-Verlag, 1988, pp. 151–218. (NATO ASI Series F, Volume 55.)

Context

These are Bird's Marktoberdorf Summer School lectures—an extended, fully worked treatment spanning 68 pages across five lectures. Each lecture poses a problem, develops the necessary theory, and solves the problem by calculation. This is the most detailed and pedagogically complete presentation of the laws, and the direct precursor to the 1989 journal paper.

Structure

Lecture 1: Lists

Covers all the definitions and laws from PRG-56, plus several extensions:

- **Multiple forms of Horner's Rule:** The basic form, a variant for non-empty lists (without requiring $\mathrm{id}_\oplus$), and a form involving map:

  $$\oplus/ \circ (\otimes/ \circ f*) \cdot \mathrm{tails} = (@) \rightarrow e$$

  where $a \mathbin{@} b = (a \mathbin{\widetilde{\otimes}} f\ b) \oplus e$.

- **Segment decomposition theorem:** A general scheme that combines promotion, Horner's Rule, and the accumulation lemma:

  If $S = \oplus/ \circ f* \circ \mathrm{segs}$ and $T = \oplus/ \circ f* \circ \mathrm{tails}$, and T can be expressed as $T = h \circ (@) \rightarrow e$, then:

  $$S = \oplus/ \circ h* \circ \mathrm{scanl}\ (@)\ e$$

Lecture 2: Homomorphisms

Extends the homomorphism lemma and proves the identity lemma:

$$h = \oplus/ \circ f* \iff h\ [a] = f\ a \text{ and } h(x +\!\!+ y) = h(x) \oplus h(y)$$

Includes worked examples: length, reverse, sort (as homomorphisms).

Lecture 3: Arrays

Generalises the entire theory to two-dimensional arrays:

- **Array accumulations:** Top accumulation and left accumulation operators.
- **Orthogonal accumulation rules:** Laws for composing accumulations along different axes.
- **Array Horner's Rule:** A two-dimensional version requiring an additional condition ("abiding").

Lecture 4: Segments (of arrays)

Applies the laws to array segment problems.

Lecture 5: Trees

Extends to tree structures with:

- **Tree reductions** and **tree accumulations** (upwards and downwards).
- **Tree accumulation lemma.**
- **Tree homomorphisms.**

Key Applications

- **Maximum segment sum (MSS):** The derivation from $O(n^3)$ to $O(n)$ that later appeared in the 1989 paper.
- **Maximum segment product:** A harder variant requiring a pair of tracked values (maximum and minimum products), solved via a generalised Horner's Rule.

Significance

PRG-69 is the most comprehensive single treatment of the laws across multiple data structures (lists, arrays, trees). It is the only source that presents the full generalisation to arrays and trees alongside the list theory.

---

5. Algebraic Identities for Program Calculation (1989)

Citation: R. S. Bird. "Algebraic Identities for Program Calculation." *The Computer Journal* 32(2):122–126, April 1989.

Context

This 5-page journal paper is a condensed, polished presentation of the Maximum Segment Sum derivation from PRG-69 Lecture 1. It is the most widely cited single source for the BMF laws in their "standard" form. Bird deliberately uses the notation of Bird & Wadler's *Introduction to Functional Programming* (1988) rather than squiggles, to maximise accessibility.

Laws Used (Numbered Equations 1–13)

The paper contains 13 numbered equations. The named laws deployed are:

1. **Definitions** (eqs. 1–4): map, foldl, scanl, inits, tails, segs, concat.
2. **Map promotion** (eq. 5):

$$\text{map } f \circ \text{concat} = \text{concat} \circ \text{map } (\text{map } f)$$

3. **Fold promotion** (eq. 6):

$$\text{foldl } (\oplus)\ e \circ \text{concat} = \text{foldl } (\oplus)\ e \circ \text{map } (\text{foldl } (\oplus)\ e)$$

4. **Horner's Rule** (eq. 9): Expressed as a transformation of $\oplus/ \circ \otimes/ * \circ$tails into a single fold, under the distributivity condition.
5. **Scan Lemma / Accumulation Lemma** (eq. 11):

$$\text{scanl } (\oplus)\ e = \text{map } (\text{foldl } (\oplus)\ e) \circ \text{inits}$$

6. **Homomorphism Lemma** (eq. 7):

$$h = (\oplus/) \circ (f*) \iff h(x +\!\!+ y) = h(x) \oplus h(y) \text{ and } h\ [a] = f\ a$$

7. **Fold–scan fusion law** (eq. 13):

$$\text{foldl } (\oplus)\ a \circ \text{scanl } (\otimes)\ b = \text{fst} \circ \text{foldl } (\odot)\ (a \oplus b,\ b)$$

   where $\odot$ is a suitably defined ternary combination of $\oplus$ and $\otimes$. This fuses a fold composed after a scan into a single fold, eliminating the intermediate list.

   Bird notes: *"There are a number of other identities concerning scanl of which we cite just one."*

The MSS Derivation

The paper derives the linear-time Maximum Segment Sum algorithm in a chain of equational steps:

$$
\begin{aligned}
\text{mss} &= \max / \circ \text{sum} * \circ\text{segs} && O(n^3) \\
&= \max / \circ \text{sum} * \circ\text{concat} \circ \text{map tails} \circ \text{inits} && \text{defn of segs} \\
&= \max / \circ (\max / \circ \text{sum} * \circ\text{tails}) * \circ\text{inits} && \text{map \& reduce promotion} \\
&= \max / \circ ((@) \rightarrow 0) * \circ\text{inits} && \text{Horner's Rule, } a \mathbin{@} b = (a+b) \uparrow 0 \\
&= \max / \circ \text{scanl } (@)\ 0 && \text{scan lemma} \\
& && O(n)
\end{aligned}
$$

Significance

This is the "canonical" reference that most subsequent authors cite when invoking the BMF laws. Its brevity (5 pages) and accessibility (standard Haskell-like notation) made it far more widely read than the longer monographs. The fold–scan fusion law (eq. 13) is particularly important for practical program optimisation.

---

6. The Algebra of Programming (1996)

Citation: R. Bird and O. de Moor. *The Algebra of Programming.* International Series in Computing Science, Volume 100. Prentice-Hall, 1996 (dated 1997).

Context

This is the comprehensive book-length treatment, co-authored with Oege de Moor. It represents the culmination of a decade of BMF development. The major intellectual shift from the earlier work is the move from *functions* to *relations* as the primary mathematical framework, and the adoption of *category theory* (functors, natural transformations, initial algebras) as the organising principle.

Structure and Scope

The book is organised around increasingly powerful algebraic structures:

Part I: Sets and Categories

- Categories, functors, natural transformations.
- Products, coproducts, initial and terminal objects.

Part II: Datatypes

- **Initial algebras and catamorphisms** (generalised folds): For any functor F with initial algebra $(\mu F, \mathrm{in})$, the unique F-algebra homomorphism from μF to any F-algebra (A, φ) is the catamorphism $(\!|\varphi|\!)$, satisfying the universal property:

$$h = (\!|\varphi|\!) \iff h \circ \mathrm{in} = \varphi \circ F\ h$$

- **Anamorphisms** (generalised unfolds) and **hylomorphisms** (unfold then fold).
- **Paramorphisms** (folds with access to the original structure).

Part III: Relations

- **Relational catamorphisms** and their laws.
- **Relational division** (a generalisation of residuation and weakest prespecification).
- **Power allegories** as the categorical framework.

Key Laws Generalised

All the laws from PRG-56/1989 are restated in the categorical/relational setting:

1. **Fusion law for catamorphisms:**

$$f \circ (\!|\varphi|\!) = (\!|\psi|\!) \Longleftarrow f \circ \varphi = \psi \circ F\ f$$

2. **Banana-split theorem** (generalised fold–scan fusion): Computes two catamorphisms in a single pass:

$$\langle (\!|\varphi|\!),\ (\!|\psi|\!) \rangle = (\!|\langle \varphi \circ F\ \pi_1,\ \psi \circ F\ \pi_2 \rangle|\!)$$

3. **Acid rain theorem** (build/fold fusion, `foldr/build`): Eliminates intermediate data structures.
4. **Accumulation and scan** as natural transformations on initial algebras.
5. **Greedy and thinning theorems** for optimisation problems: conditions under which a greedy algorithm produces an optimal solution.
6. **Dynamic programming** conditions: when greedy fails, conditions for efficient memoised computation.

Significance

"Algebra of Programming" is the most general and most powerful treatment of the laws. It provides categorical proofs that work for any inductively defined datatype, not just lists. However, as Jeremy Gibbons noted, it contains "a lot of laws" and is a difficult read. The relational framework, while elegant, proved too complex for wide adoption, and both Bird and the community eventually retreated to a purely functional approach.

---

7. Generic Functional Programming with Types and Relations (1996)

Citation: R. Bird, O. de Moor, and P. Hoogendijk. "Generic Functional Programming with Types and Relations." *Journal of Functional Programming* 6(1):1–28, 1996.

Context

This paper bridges the gap between the list-specific laws of PRG-56/1989 and the fully categorical treatment of "Algebra of Programming." It asks: what do the scan lemma and Horner's Rule mean for *non-linear* datatypes like trees?

Key Contributions

Datatype-Generic "tails" and "inits"

For lists, tails returns all suffixes and inits returns all prefixes. For a general datatype (e.g., a tree):

- A **tail** (or "subtree") of a data structure is a subterm.
- An **initial segment** is the structure with some subterms replaced by the empty structure.

The paper introduces the **labelled variant** of a datatype: given a type constructor F, the labelled variant provides exactly one label per node (regardless of the original labelling). This gives a datatype-generic notion of "the tree of all subterms."

Datatype-Generic Scan Lemma

$$\text{scans} = \text{map}\ (\text{fold}\ \varphi) \circ \text{subtrees}$$

where subtrees produces the labelled variant tree whose node labels are the subterms of the original tree, and scans replaces each node's label with the fold of the subtree rooted there.

This generalises $\text{scanl}\ (\oplus)\ e = \text{map}\ (\text{foldl}\ (\oplus)\ e) \circ \text{inits}$ from lists to arbitrary datatypes.

Datatype-Generic Horner's Rule

The paper also generalises Horner's Rule. Given a "sum" algebra $\oplus/$ and a "product" algebra $\otimes/$, under suitable distributivity conditions:

$$\oplus/ \circ \text{map}\ (\otimes/) \circ \text{inits} = \text{fold}\ (@)$$

for a suitably defined @ combining $\oplus$ and $\otimes$.

Significance

This paper shows that the laws discovered for lists are not list-specific accidents but instances of general algebraic principles that hold for any inductively defined datatype. The datatype-generic scan lemma is a particularly beautiful result, and Gibbons later expanded on it in his work on generic downwards accumulations.

---

8. Algorithm Design with Haskell (2020)

Citation: R. S. Bird and J. Gibbons. *Algorithm Design with Haskell*. Cambridge University Press, 2020.

Context

This book is Bird's final major work, co-authored with Jeremy Gibbons. It represents a deliberate return to the original vision of "a textbook on algorithmics"—the imagined book that motivated the entire BMF project from the 1970s onwards. Unlike "Algebra of Programming," it uses **plain Haskell** with no squiggles, no category theory, and only a minimal excursion into nondeterminism.

Structure

The book covers algorithm design techniques, each presented as a calculational derivation:

- **Divide and conquer** (including MSS, maximum non-segment sum).
- **Greedy algorithms** (including Huffman coding, scheduling).
- **Thinning** (when greedy fails, a thinning step prunes the search space).
- **Dynamic programming** (systematic memoisation of recursive specifications).
- **Exhaustive search** (backtracking, branch and bound).

Laws Presented

All the core laws are re-derived in Haskell notation:

1. **Map fusion:** `map f . map g = map (f . g)`
2. **Fold fusion:** `f . foldr g a = foldr h b` provided `f a = b` and `f (g x y) = h x (f y)` for all `x`, `y`.
3. **Scan lemma:** `scanl f e = map (foldl f e) . inits`
4. **Fold–scan fusion:** `foldl g a . scanl f b = fst . foldl h (a, b)` for suitably defined `h`.
5. **Horner's Rule:** Expressed as a derivation pattern for problems of the form `maximum . map (foldl f e) . tails`.
6. **Accumulation lemma and its converse.**
7. **Greedy conditions:** The greedy algorithm produces an optimal solution when (a) the cost function is monotonic, and (b) a greedy choice can always be extended to an optimal solution.
8. **Thinning conditions:** When greediness is too aggressive, a thinning step retains a manageable subset of candidates.

Notation

Where the 1989 paper used:

$$\max / \circ \text{sum} * \circ \text{segs}$$

this book writes:

```
maximum . map sum . segs
```

The laws are the same; only the surface syntax has changed.

Significance

"Algorithm Design with Haskell" closes the circle opened in the 1970s. It is the accessible textbook on algorithmics that Bird and Meertens envisioned, written in a notation that a broad audience can read. It demonstrates that the algebraic approach to algorithm design—deriving efficient programs from clear specifications by equational reasoning—is viable and practical without any special notation or advanced mathematics.

Bird passed away in 2022. This book stands as his final statement on the subject.

The Evolution of Notation for Fold and Scan

The symbols used for reduce, fold, and scan changed substantially across Bird's publications and the broader BMF community. This section traces that evolution.

Phase 1: APL Origins (1962–1978)

Kenneth Iverson's *A Programming Language* (1962) introduced:

APL	Name	Meaning
`+/`	Reduce	Sum of a vector (fold right with +)
`+\`	Scan	Running sums (prefix sums)

The slash `/` for reduce and backslash `\` for scan became standard in APL. In 1978, Iverson's "Operators and Functions" added the *commute* operator (written with the tilde-dieresis glyph), which swaps the arguments of a dyadic function—a precursor to Bird's tilde notation for flip.

APL's reduce and scan are *symmetric*: they assume the operator is associative and make no left/right distinction.

Phase 2: Meertens' Algorithmics (1984–1986)

Lambert Meertens adopted APL's slash directly into his "Algorithmics" notation at CWI Amsterdam:

Symbol	Meaning
$\oplus/$	Symmetric reduce (from APL's `+/`)
$\oplus\backslash$	Symmetric scan (from APL's `+\`)

These still require associativity and have no left/right distinction.

Phase 3: The Squiggol Period (1985–1988)

Bird and Meertens, working within IFIP WG,2.1, introduced *directed* folds and scans. The key innovation was adding **directional arrows over the slash** to indicate processing order. According to Gibbons' "The School of Squiggol" (2020):

> *"... arrows for directed folds and scans ('$\rightarrow$ /' and '$\rightarrow$ //', now called 'foldl' and 'scanl' in Haskell)..."*

To preserve the historical glyph shapes, the exact fold and scan symbols are shown below as readable PRG-69 facsimiles rather than being squeezed into tiny table cells:

Directed Folds	Directed Scans
$\oplus \not{\rightarrow} e$	$\oplus \not{\rightarrow\!\!\!\rightarrow} e$
$\oplus \not{\leftarrow} e$	$\oplus \not{\leftarrow\!\!\!\leftarrow} e$
upper = left-to-right, lower = right-to-left	upper = left-to-right, lower = right-to-left
modern: `foldl (⊕) e` / `foldr (⊕) e`	modern: `scanl (⊕) e` / `scanr (⊕) e`

Other symbols from the same period were still written compositionally: $\oplus/$ for symmetric reduce, $f*$ for map, and $\widetilde{\oplus}$ for flip.

The design principle: **single slash** $/$ = fold (one result), **double slash** $/\!\!/$ = scan (all intermediate results). The arrow indicates processing direction: left-to-right on the upper fold/scan form, right-to-left on the lower one.

This is the notation used in PRG-56 (1986), PRG-64 (1987), and PRG-69 (1988), though the printed monographs were typeset with 1980s tools and the exact glyph rendering varies.

Phase 4: The Retreat from Squiggles (1988–1989)

Bird and Wadler's textbook *Introduction to Functional Programming* (1988) abandoned the squiggly symbols in favour of alphabetic Miranda-style names. The 1989 "Algebraic Identities" paper followed suit, explicitly stating:

> *"In order to make the material as accessible as possible, we shall use the notation for functional programming described by Bird and Wadler."*

1989 Notation	**Meaning**
`foldl` $(\oplus)$ `e`	left fold
`foldr` $(\oplus)$ `e`	right fold
`scanl` $(\oplus)$ `e`	left scan
`scanr` $(\oplus)$ `e`	right scan
`map f`	map

The squiggly symbols for fold/scan were never used again by Bird in published work after 1989.

Phase 5: Categorical Notation (1993–1997)

In `The Algebra of Programming'' (1996), Bird and de Moor replaced fold with the \emph{catamorphism} in`banana brackets," and unfold with the *anamorphism* in "lens brackets":

Symbol	**Meaning**
$(\!\mid \varphi \mid\!)$	catamorphism (generalised fold)
$[\!(\psi)\!]$	anamorphism (generalised unfold)
$[\varphi, \psi]$	hylomorphism (unfold then fold)

Scans are expressed indirectly as natural transformations on initial algebras; there is no dedicated scan symbol.

Phase 6: Plain Haskell (1998–2020)

From the second edition of Bird's textbook (1998) through "Algorithm Design with Haskell" (2020), all notation is standard Haskell: `foldl`, `foldr`, `scanl`, `scanr`, `map`. No special symbols.

Summary Timeline

Year	Source	Fold	Scan
1962	Iverson, APL	`+/`	`+\`
1978	Iverson, "Operators and Functions"	(same)	(same)
1984	Meertens, Algorithmics	$\oplus/$	$\oplus\backslash$
1986–1988	Bird, PRG-56/64/69	directed fold glyphs (see Phase 3)	directed scan glyphs (see Phase 3)
1988	Bird & Wadler, textbook	`foldl`, `foldr`	`scanl`, `scanr`
1989	Bird, "Algebraic Identities"	`foldl`, `foldr`	`scanl`, `scanr`
1996	Bird & de Moor, AoP	$(\!\mid \varphi \mid\!)$	(natural transformation)
2020	Bird & Gibbons, ADwH	`foldl`, `foldr`	`scanl`, `scanr`

The exact 1986–1988 fold/scan facsimiles are shown at readable size in Phase 3 above. The summary timeline names the notation without shrinking scanned glyphs into illegible table cells.

Note on the tilde: Bird's $\widetilde{\oplus}$ for "flip" (where $a \mathbin{\widetilde{\oplus}} b = b \oplus a$) appeared in PRG-56 (1986). Iverson had used the related glyph ⍨ (tilde-dieresis, APL glyph) for the commute operator in APL since 1978. When Iverson designed J (circa 1990), he simplified this to a bare tilde `~`. Whether Bird's notation was directly influenced by APL's or arrived independently from the mathematical convention of tilde-as-variant is unclear, but Bird was demonstrably familiar with APL (the BMF borrowed `/` for reduce directly from it).

Summary: Lineage of the Laws

The following diagram shows how the laws developed across publications:

```
Bird (1984) TOPLAS
  "Promotion and Accumulation Strategies"
  [promotion, accumulation as named strategies]
        |
        v
Bird (1986) PRG-56
  "An Introduction to the Theory of Lists"
  [ALL core list laws: map/reduce promotion, duality,
   specialisation, accumulation lemma, Horner's Rule,
   homomorphism lemma]
        |
        +-----> Bird (1987) PRG-64
        |         "A Calculus of Functions"
        |         [two general theorems unifying the laws]
        |
        v
Bird (1988) PRG-69
  "Lectures on Constructive Functional Programming"
  [full worked treatment; extensions to arrays and trees;
   segment decomposition theorem; MSS and MSP derivations]
        |
        v
Bird (1989) Computer Journal
  "Algebraic Identities for Program Calculation"
  [condensed 5-page journal version; fold-scan fusion law;
   the canonical citation]
        |
        +-----> Bird & de Moor (1996)
        |         "The Algebra of Programming"
        |         [categorical/relational generalisation;
        |          catamorphisms, banana-split, acid rain;
        |          greedy and DP theorems]
        |
        +-----> Bird, de Moor & Hoogendijk (1996)
        |         "Generic Functional Programming"
        |         [datatype-generic scan lemma and Horner's Rule]
        |
        v
Bird & Gibbons (2020)
  "Algorithm Design with Haskell"
  [modern Haskell re-derivation; no squiggles;
   the accessible textbook Bird always envisioned]
```

Appendix A: Quick Reference of Core Laws (Haskell Notation)

For convenience, here are the principal laws in modern Haskell-style notation.

L1. Map fusion:

```
map f . map g  =  map (f . g)
```

L2. Map promotion:

```
map f . concat  =  concat . map (map f)
```

L3. Fold promotion:

```
foldr f e . concat  =  foldr f e . map (foldr f e)
```

L4. Fold fusion:

```
h . foldr f a  =  foldr g b
  provided  h a = b  and  h (f x y) = g x (h y)
```

L5. Scan lemma (accumulation lemma):

```
scanl f e  =  map (foldl f e) . inits
```

L6. Last-fold identity:

```
last . scanl f e  =  foldl f e
```

L7. Fold–scan fusion:

```
foldl g a . scanl f b  =  fst . foldl h (a `g` b, b)
  where h (acc, s) x = (acc `g` (f s x), f s x)
```

L8. Horner's Rule: If $(\otimes)$ distributes backwards over $(\oplus)$:

```
foldr1 g . map (foldl f e) . tails  =  foldl h e
  where h a b = g (f a b) e
```

L9. Homomorphism lemma:

```
h = foldr1 g . map f
  iff  h [a] = f a  and  h (xs ++ ys) = h xs `g` h ys
```

L10. Duality:

```
foldl f e  =  foldr (flip f) e . reverse
```

Appendix B: Core Laws in BMF Symbolic Notation

The same laws as Appendix A, expressed in the Bird–Meertens "Squiggol" notation as used in PRG-56 (1986) and PRG-69 (1988). The fold and scan symbols use the arrow-over-slash convention confirmed by Gibbons (2020).

Symbol	Meaning
$f*$	map f (apply f to every element)
$\oplus/$	symmetric reduce by $\oplus$ (requires associativity)
$\oplus\overrightarrow{/}\,e$	left fold with seed e ($\rightarrow$ over $/$ = left-to-right; modern `foldl`)
$\oplus\overleftarrow{/}\,e$	right fold with seed e ($\leftarrow$ over $/$ = right-to-left; modern `foldr`)
$\oplus\overrightarrow{//}\,e$	left scan with seed e ($\rightarrow$ over $//$ = left-to-right accumulation; modern `scanl`)
$\oplus\overleftarrow{//}\,e$	right scan with seed e ($\leftarrow$ over $//$ = right-to-left accumulation; modern `scanr`)
$*$ (binary)	list concatenation ($+\!\!+$)
$*/$	concat (flatten; reduce by concatenation)
$[a]$	singleton list containing a
$[\,]$	empty list
$\widetilde{\oplus}$	flip of $\oplus$: $a \,\widetilde{\oplus}\, b = b \oplus a$
$\mathrm{id}_\oplus$	identity element of $\oplus$
$p\triangleleft$	filter by predicate p
$\cdot$	functional composition (Haskell's "`.`")

Design principle: single slash $/$ = fold (one result); double slash $//$ = scan (all intermediate results). The arrow indicates direction: $\rightarrow$ = left-to-right, $\leftarrow$ = right-to-left.

L1. Map fusion (distributivity over composition):

$$(f \cdot g)* \;=\; (f*) \cdot (g*)$$

L2. Map promotion (join rule for map):

$$f* \cdot */ \;=\; */ \cdot (f*)*$$

L3. Reduce promotion (join rule for reduce):

$$\oplus/ \cdot */ \;=\; \oplus/ \cdot (\oplus/)*$$

L4. Fold fusion (for right reduction):

$$h \cdot (\oplus\overleftarrow{/}\,a) \;=\; \otimes\overleftarrow{/}\,b$$

provided $h\ a = b$ and $h\ (x \oplus y) = (h\ x) \otimes y$ for all x, y.

L5. Accumulation lemma (scan lemma):

$$\oplus\overrightarrow{//}\,e \;=\; (\oplus\overrightarrow{/}\,e)* \cdot \mathit{inits}$$

L6. Last-fold identity:

$$\mathit{last} \cdot \overrightarrow{\oplus//}\, e \;=\; \overrightarrow{\oplus/}\, e$$

L7. Fold–scan fusion:

$$(\overrightarrow{\oplus/}\, a) \cdot (\overrightarrow{\otimes//}\, b) \;=\; \pi_1 \cdot \overrightarrow{\odot/}\, (a \oplus b,\ b)$$

where $(r, s) \odot x = (r \oplus (s \otimes x),\ \ s \otimes x)$.

L8. Horner's Rule:

If $\otimes$ distributes backwards over $\oplus$ (i.e., $(a \oplus b) \otimes c = (a \otimes c) \oplus (b \otimes c)$), then:

$$\oplus/ \cdot (\otimes/) * \cdot \mathit{tails} \;=\; \overleftarrow{@/}\, e$$

where $e = \mathrm{id}_{\oplus}$ and $a \mathbin{@} b = (a \otimes b) \oplus e$.

L9. Homomorphism lemma:

$$h \;=\; (\oplus/) \cdot (f*) \quad \Longleftrightarrow \quad h\ [a] = f\ a \ \text{ and } \ h\ (x * y) = h\ x \oplus h\ y$$

L10. Duality lemma:

$$\overrightarrow{\oplus/}\, e \;=\; (\overleftarrow{\widetilde{\oplus}/}\, e) \cdot \mathit{reverse}$$

where $a \mathbin{\widetilde{\oplus}} b = b \oplus a$.